Lecture 22

Using SAM/BAM as data storage. Unaligned BAM.

The PAST haunts us

It used to be that we had **one file**. That one, very precious, expensive **sequencing run**

This was long-long time ago ... perhaps 2012 ...

The rules we use today were made then - to solve those problems.

Yesterday we had to set up a cacao infection resistance RNA-Seq study with `7` time points, over `2` genotypes each measured `4` times.

```
7 x 2 x 4 = 56 samples
```

SAM/BAM cottage industry 

You will often find that the SAM format exhibits both traits:

- incredibly useful
- woefully inadequate

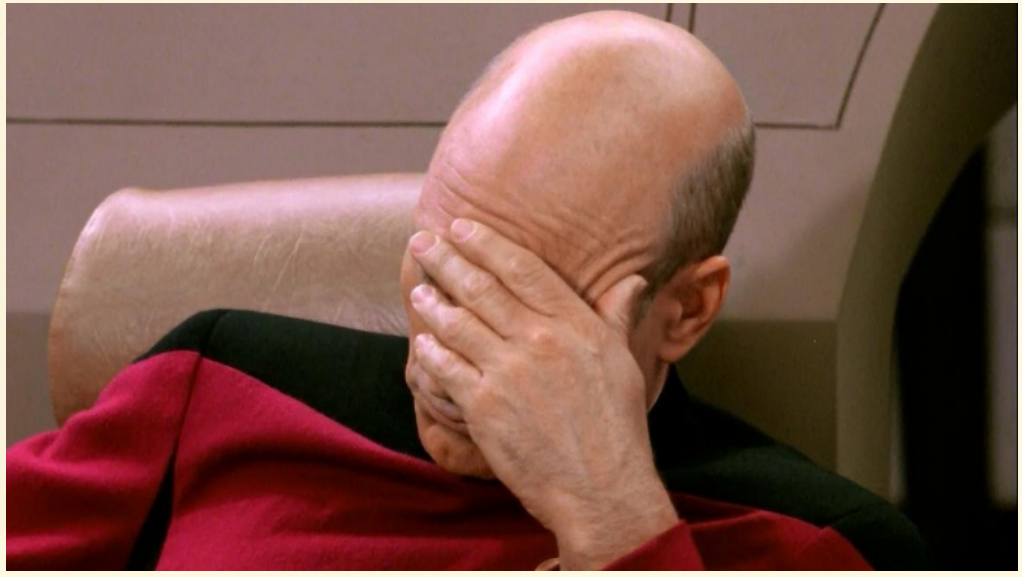
All within the same project. Essential ingredients seem to be missing.

An entire "cottage industry" of tools sprung up around it. Everyone tries to make tools that fill some of the gaps.

The Picard Programs 🚀

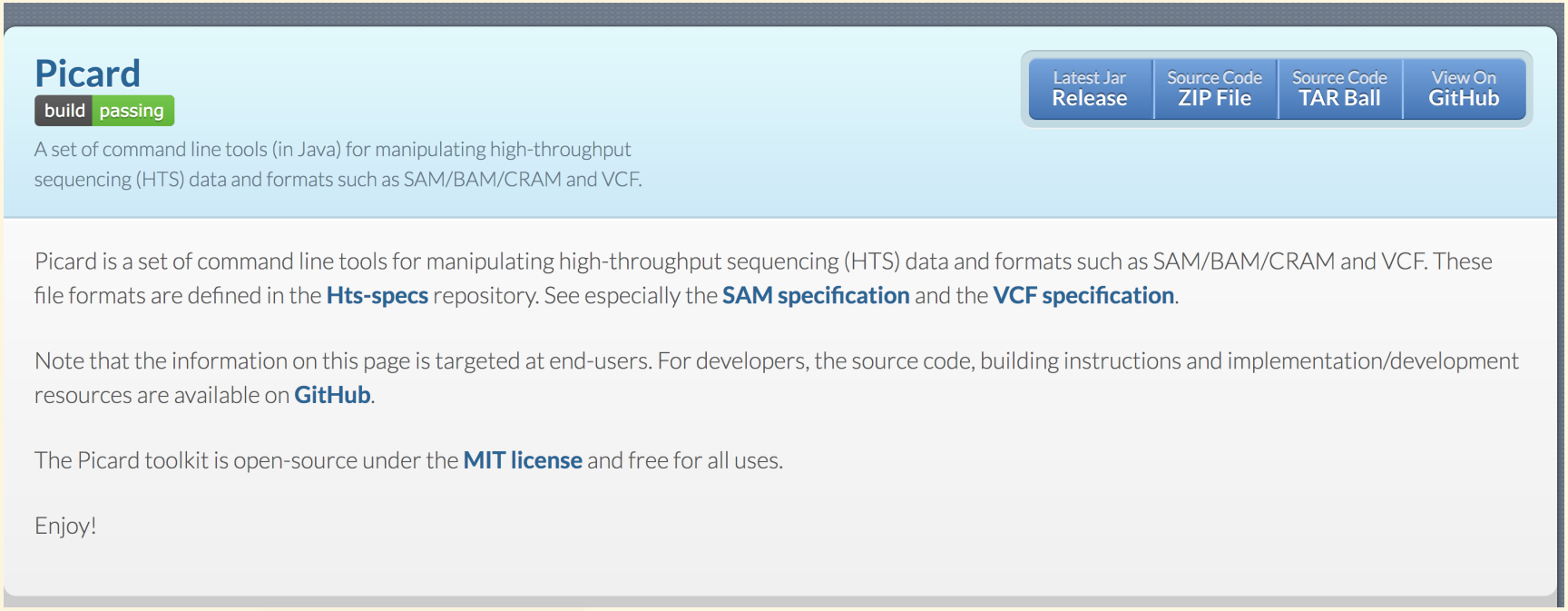
The name of the tool is a riff on **Star Trek the Next Generation's** captain **Picard**.

The "most famous" meme of Jean Luc Picard:



Embrace the feeling.

Picard provides ways to "process" SAM files



Picard

build passing

A set of command line tools (in Java) for manipulating high-throughput sequencing (HTS) data and formats such as SAM/BAM/CRAM and VCF.

Latest Jar Release | Source Code ZIP File | Source Code TAR Ball | View On GitHub

Picard is a set of command line tools for manipulating high-throughput sequencing (HTS) data and formats such as SAM/BAM/CRAM and VCF. These file formats are defined in the **Hts-specs** repository. See especially the **SAM specification** and the **VCF specification**.

Note that the information on this page is targeted at end-users. For developers, the source code, building instructions and implementation/development resources are available on **GitHub**.

The Picard toolkit is open-source under the **MIT license** and free for all uses.

Enjoy!

A mixture of useful and peculiar/particular/odd functionality.

Reading the Picard docs makes you realize what kinds of information a SAM file might contain.

Picard is a collection of many programs



See also the Picard website.

Picard usage example: DownsampleSam

```
wget http://data.biostarhandbook.com/bam/demo.bam
```

Get help on the program:

```
picard DownsampleSam -h
```

It uses Java style "flags".

```
picard DownsampleSam I=demo.bam O=small.bam P=0.1
```

output:

```
DownsampleSam   Finished downsampling.
DownsampleSam   Kept 2985 out of 31571 reads (9.45%).
```

Picard is a "better/different/quirky" samtools

picard is developed at the Broad Institute. Has a lot more functionality. Some very peculiar.

samtools was also created at the Broad Institute (Heng Li) but today is being developed by the European Bioinformatics Institute.

picard uses the same philosophy as the **GATK** variant caller. It has the same type of quirkiness to it. They integrate "well" - require the same coping skills.

If you are using human DNA sequencing data then **picard + GATK** is the recommended practice.

Is your BAM file hiding something?

Check the picard programs.

One of them might just

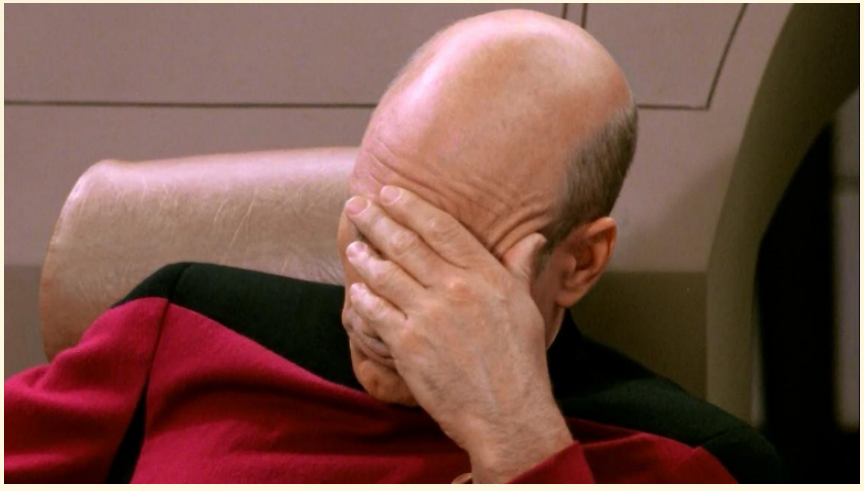
make your BAM file confess.

The unaligned BAM file

What is **SAM/BAM**? **Sequence Alignment Maps** were designed to represent and store **alignment information**.

HEY PEOPLE! HERE IS A GREAT IDEA!

Lets use **SAM/BAM** files for **unaligned** sequences.



Seriously?

Why on Earth would you do that?

Desperate times call for desperate measures

There is **no** sequence data storage format that could store the sample information **within** the data.

Many encode the sample info into the file name. But that is a bad idea:

- There is no standard naming.
- Too easy to change accidentally.
- A file name is not the place to encode lots of information.

For BAM files **read groups** were invented.

Revisiting Read groups

The SAM header carries the readgroup definition

```
@HD     VN:1.3  SO:coordinate
@SQ     SN:AF086833.2   LN:18959
@RG     ID:p1   SM:Ebola1       LB:patient_101
@RG     ID:p2   SM:Ebola2       LB:patient_102
```

Tags each alignment with sample information.

- `ID`: unique id, `SM`: sample name, `LB`: library name

Then an optional attribute of the alignment can indicate the sample

```
MD:Z:81C19  MC:Z:101M  AS:i:96  XS:i:0   RG:Z:p2
```

This way a single BAM file may contain alignments for ALL samples. Tens, hundreds, thousands of them.

The rationale for UNALIGNED BAM

FASTQ information can be fully stored in the BAM.

Using readgroups it could also be tagged by sample.

There is only one format to deal with.

Other information could be encoded in other tags.

Nagging suspicion: Is this a good idea really?

What is not so great about unaligned BAM?

Hard to tell what is stored in the file. You have to look into the header.

The presence of a readgroup in the header does not mandate that the data is present.

No way to efficiently extract a sample by tag.

It is time consuming to locate a tag.

This will cause even more problems in the future.

No more FASTQ! BAM files all way.



Alignment then goes from unaligned to aligned BAM.

Creating an unaligned BAM file

```
# Get reads
fastq-dump -X 10000 --split-files SRR1972739

# Set up shortcuts.
R1=SRR1972739_1.fastq
R2=SRR1972739_2.fastq

# Convert FASTQ to BAM file.
picard FastqToSam F1=$R1 F2=$R2 O=out.bam
                                RG=joe LB=sunday SM=first
```

view the header with `samtools view -H out.bam`

```
@HD     VN:1.5  SO:queryname
@RG     ID:joe  SM:first         LB:sunday
```

You can also tag during alignment:

You can tag during alignment:

```
TAG='@RG\tID:xyz\tSM:Ebola\tLB:patient_100'
```

Add the tags during alignment

```
bwa mem -R $TAG $REF $R1 $R2 | samtools sort > bwa.bam
```

Add readgroups into a BAM file

```
TAG='@RG\tID:A\tSM:B\tLB:C'

samtools addreplacerg -m overwrite_all -r $TAG demo.bam
                                                -O BAM > new.bam
```

See the headers:

```
samtools view -H new.bam
```

The SAM header carries the readgroup definition:

```
@HD     VN:1.3  SO:coordinate
@SQ     SN:AF086833.2   LN:18959
@RG     ID:A    SM:B    LB:C
```

Each alignment is now tagged

```
samtools view new.bam | cut -f 14-17 | head -1
```

The format keeps track of the origins of each read:

```
MC:Z:101M    AS:i:96    XS:i:0    RG:Z:A
```

The unique id `A` identifies sample name `B` and library `C` from the header.

Many tools rely on the presence of read groups.

It is an essential skill to understand them.

From BAM to FASTQ

Most tools don't support unaligned BAM as input.

To run the tool you need to convert from unaligned BAM to FASTQ

```
samtools fastq unaligned.bam -1 read1.fq -2 read2.fq
```

PacBIO instument produces a BAM file

The PacBIO instrument produces an unaligned BAM as a sequencing result. No FASTQ file is provided:

```
wget http://data.biostarhandbook.com/bam/pacbio.bam
```

What kinds of BAM tags does it have:

```
samtools view pacbio.bam | cut -f 12,14,16-20 | head -5

cx:i:0  np:i:1  qe:i:29331 qs:i:28179 rq:f:0.8  sn:B:f,7.1
cx:i:0  np:i:1  qe:i:8471  qs:i:0  rq:f:0.8  sn:B:f,6.8619
cx:i:0  np:i:1  qe:i:1029  qs:i:0  rq:f:0.8  sn:B:f,7.8491
cx:i:0  np:i:1  qe:i:10060 qs:i:0  rq:f:0.8  sn:B:f,6.4855
cx:i:2  np:i:1  qe:i:13985 qs:i:0  rq:f:0.8  sn:B:f,7.2136
```

It keeps getting better (sarcasm)

```
samtools view pacbio.bam | cut -f 13 | head -1


ip:B:C,86,35,42,17,10,25,11,17,38,9,17,9,52,1,2,23,37,2,15
8,132,124,43,15,25,14,94,72,10,8,50,16,26,2,22,1,23,13,4,5
35,1,1,51,5,11,6,10,9,35,8,76,22,27,82,32,2,35,32,26,3,26,
16,74,16,25,35,10,4,35,74,13,1,16,13,47,5,78,43,31,56,42,9
255,6,54,28,14,37,106,54,45,2,31,2,68,28,30,12,5,56,37,80,
36,9,72,4,80,53,142,36,12,7,22,68,63,1,138,60,7,35,9,8,55,
63,16,31,2,20,2,2,6,132,144,11,96,26,78,42,13,1,4,3,21,13,
19,9,45,13,39,55,23,22,19,110,7,22,1,8,4,12,1,41,12,28,57,
66,126,4,33,38,29,8,3,2,45,152,57,64,15,36,61,106,48,22,41
25,29,2,255,14,32,80,44,39,37,14,12,48,35,80,1,140,255,16,
160,6,6,21,34,19,17,116,4,13,46,49,9,25,24,37,16,18,255,38
9,47,132,66,132,9,82,51,68,28,134,11,10,16,88,19,55,60,47,
...
```

The Gods Must Be Crazy

